

O ważnych decyzjach warto pamiętać

ADR-y dla developerów

Marek Gawryszewski

C/Embedded/Hardware User Group, 20.05.2026

Settings

- ~45 min prezentacji
- duże Q & A na koniec
- szybkie pytania w trakcie, większe parkujemy na koniec

Problem

- „Dlaczego używamy akurat MySQL?”
- Tylko *Wojtek* wie, "dlaczego"
- Ważne decyzje podejmowane "intuicyjnie"
- Ważne decyzje podejmowane ad-hoc

Czym jest ADR?

ADR

ADR (Architecture Decision Record): *krótki* dokument opisujący **jedną** ważną decyzję dotyczącą architektury, jej kontekst oraz konsekwencje dla projektu.

Dokument zawiera sensowne minimum informacji.

Najpopularniejsza struktura:

- tytuł,
- rozważane rozwiązania,
- decyzja,
- konsekwencje.

ADR mówi o dwóch rzeczach:

1. Jaka **decyzja** została podjęta
2. W jaki był jej **kontekst**

Po co to robić?

- archiwum ważnych decyzji
- zapisany kontekst decyzji
- dobre uzupełnienie dla HLD
- ułatwiony onboarding/take over
- Pisanie ADR wspomaga proces decyzyjny.

Przykład

Architecture Decision Record: Rust programming language

Korzyści

Zespół

- wiedza instytucjonalna
- ułatwiony onboarding
- utrzymywany kontekst
- dupokrytka (decyzje klepnięte przez Archa/PO)
- pisanie wspomaga myślenie

Architekci

- Historia zmian w jednym miejscu
- Forma dyskusji z biznesem/dev/sec/ops

Managerowie

- procesowa forma podejmowania decyzji
- decyzje projektowe/architektoniczne mają uzasadnienie

Dobry ADR

- napisany w pliku tekstowym (markdown)
- użyteczne minimum informacji
- struktura na bazie szablonu
- raz napisany, nie jest zmieniany
- standard diagramów: mermaid, plantuml

Czym ADR nie jest

- dokumentacja systemu / HLD
- change log
- release notes

Kiedy nie stosować

- małe projekty
- projekty w głębokim maintenance

Use AI, Luke!

- AI świetnie współpracuje z Markdown
- Agent pomoże napisać ADR
- AI podsumuje poprzednie decyzje
- AI pomoże z nudną częścią, ale to białko musi podjąć decyzję!

Jak zacząć?

- Wybierz decyzje, które mają duży wpływ na architekturę lub trudno je odwrócić
- Użyj prostego szablonu wspólnego dla całego projektu
- Trzymaj ADR-y w repozytorium
- Użyj Pull Request do review
- Zaczniij od kilku najważniejszych decyzji, nie od pełnej historii systemu

Jak zespół pracuje z ADR?

1. Ktoś w zespole identyfikuje istotną decyzję i tworzy ADR
2. Zespół czyta dokument i komentuje go podczas review
3. ADR zostaje przyjęty, odrzucony albo wraca do dopracowania

Jak utrzymać ADR-y

- wszystkie ADR-y są w repo o najbliższym kontekście
- Zaakceptowanych ADR-ów nie powinno się usuwać ani zmieniać
- Zmiana decyzji to nowa decyzja
- Warto linkować do innych ADR-ów, Jira, Confluence etc.

Czego unikać?

- ADR-a dla każdej drobnej decyzji
- Długie dokumenty
- ADR-y poza repozytorium
- Usuwanie starych decyzji z repozytorium

Take away

- ADR-y może tworzyć każdy, na każdym etapie projektu
- decyzja, nie design
- markdown
- KISS!

Q&A

Linki

- [Michael Nygard: Documenting Architecture Decisions](#)
- [Martin Fowler: Architecture Decision Record](#)